



贪吃蛇游戏设计汇报

小组成员：



时间：





北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

目录

CONTENTS

1

基本情况

2

设计与实现

3

测试与结论

4

总结与思考



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

1

基本情况

题目内容

设计并实现一款贪吃蛇游戏，在游戏中玩家可以控制贪吃蛇寻找食物，每吃一口食物蛇身就会变长并且获得游戏分数，当贪吃蛇碰到墙壁、障碍物或者咬到自己的身体则游戏结束。



题目要求及约束条件

在遵循上述游戏规则的前提下，玩家可以使用键盘控制贪吃蛇移动方向。在屏幕中可以随机产生食物，并且贪吃蛇吃到食物后蛇身变长同时获得一定分数。贪吃蛇的头不能碰到自己的身体，也不能碰到墙壁，否则游戏结束。游戏分为闯关模式和无限模式供玩家选择，并提供暂停\继续、加速\减慢蛇身移动等操作以增强玩家的游戏体验。

1



2



3



4



5



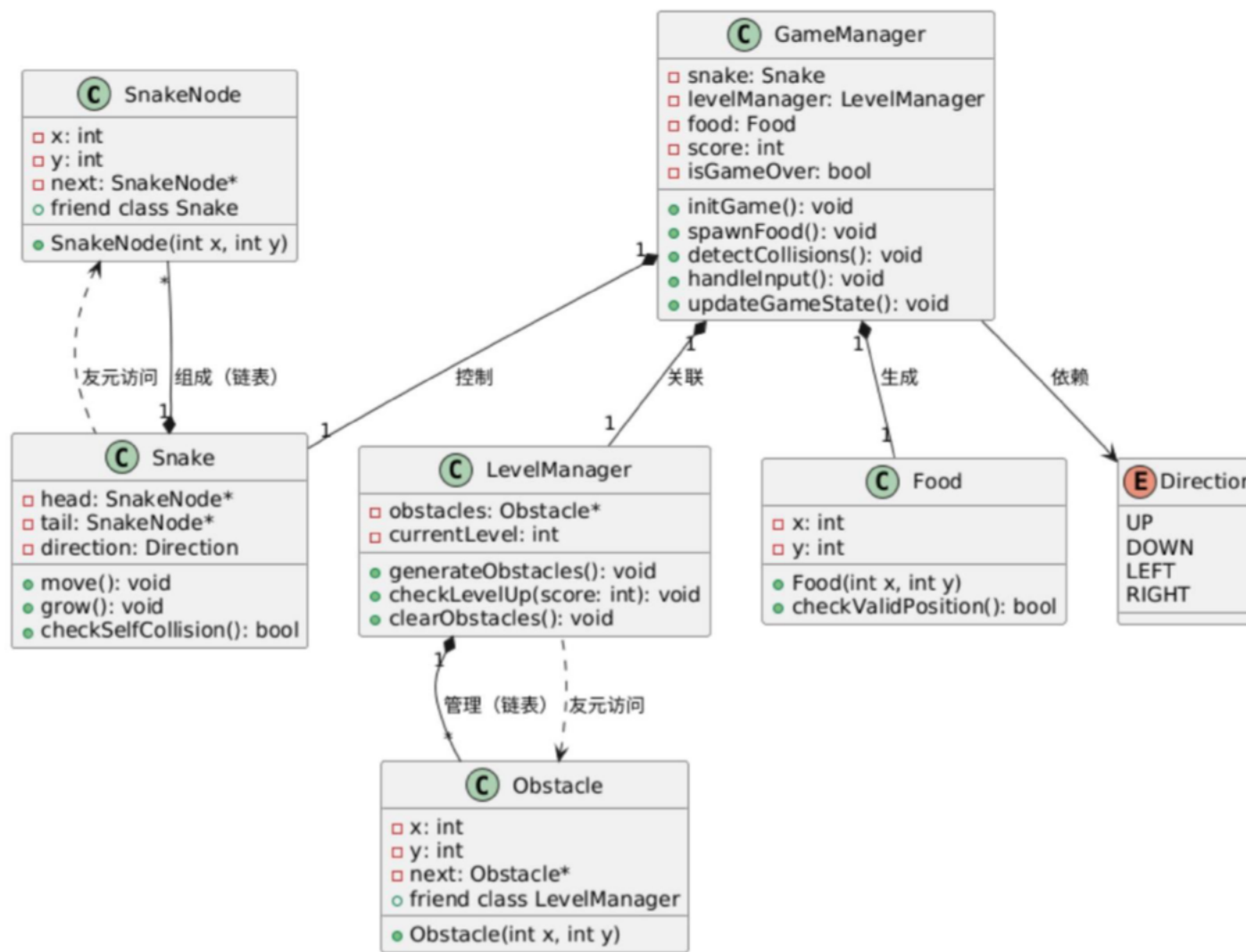


北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

2

设计与实现

贪吃蛇是一款简单有趣的休闲益智类游戏，自从问世以来就深受玩家的欢迎。我们组基于C++编程语言编写了一款贪吃蛇游戏，支持**无限模式与闯关模式**供玩家选择。程序采用**链式队列**数据结构管理**蛇身和障碍物**，实现贪吃蛇的**蛇身生成、位置判定、移动控制和碰撞判定**等功能。与此同时，程序通过Windows控制台API实现**控制台交互**，进而实现控制台的**光标定位、颜色设置、键盘控制**等功能，进一步提升玩家的游戏体验感。



蛇身节点
(SnakeNode)类:
存储每一节蛇身的
坐标位置以及指向
下一节点的指针。
同时将相应的函数
设为友元, 实现蛇
身创建、蛇身移动
、碰撞检测等操作
。

```
// 蛇节点类
class SnakeNode {
private:
    int x;
    int y;
    SnakeNode* next;

    friend void moveSnake(int newX, int newY);
    friend int biteself();
    friend void createfood();
    friend void initsnake();
    friend void snakemove();
    friend void cantcrosswall();
    friend void createObstacles();
    friend int hitObstacle();
    friend void gamecircle();

public:
    SnakeNode(int px, int py, SnakeNode* pnext = nullptr)
    { : x(px), y(py), next(pnext) {}
};
```

障碍物
(Obstacle)类:
存储每一个障
碍物的坐标位
置以及指向下
一个障碍物的
指针。同时将
创建障碍物以
及碰撞检测等
函数设为友元
，保证游戏的
正常进行。

```
// 障碍物类
class Obstacle {
private:
    int x;
    int y;
    Obstacle* next;

    friend void createObstacles();
    friend int hitObstacle();
    friend void gamecircle();

public:
    Obstacle(int px, int py, Obstacle* pnext = nullptr)
    { : x(px), y(py), next(pnext) {}
};
```

数据结构

链式队列

程序在创建蛇身运用到进队操作，蛇身移动和吃到食物延长蛇身的时候同时运用进队和出队操作。

单向链表

程序在创建障碍物时运用单向链表，有利于后续关卡障碍物个数增加的操作。

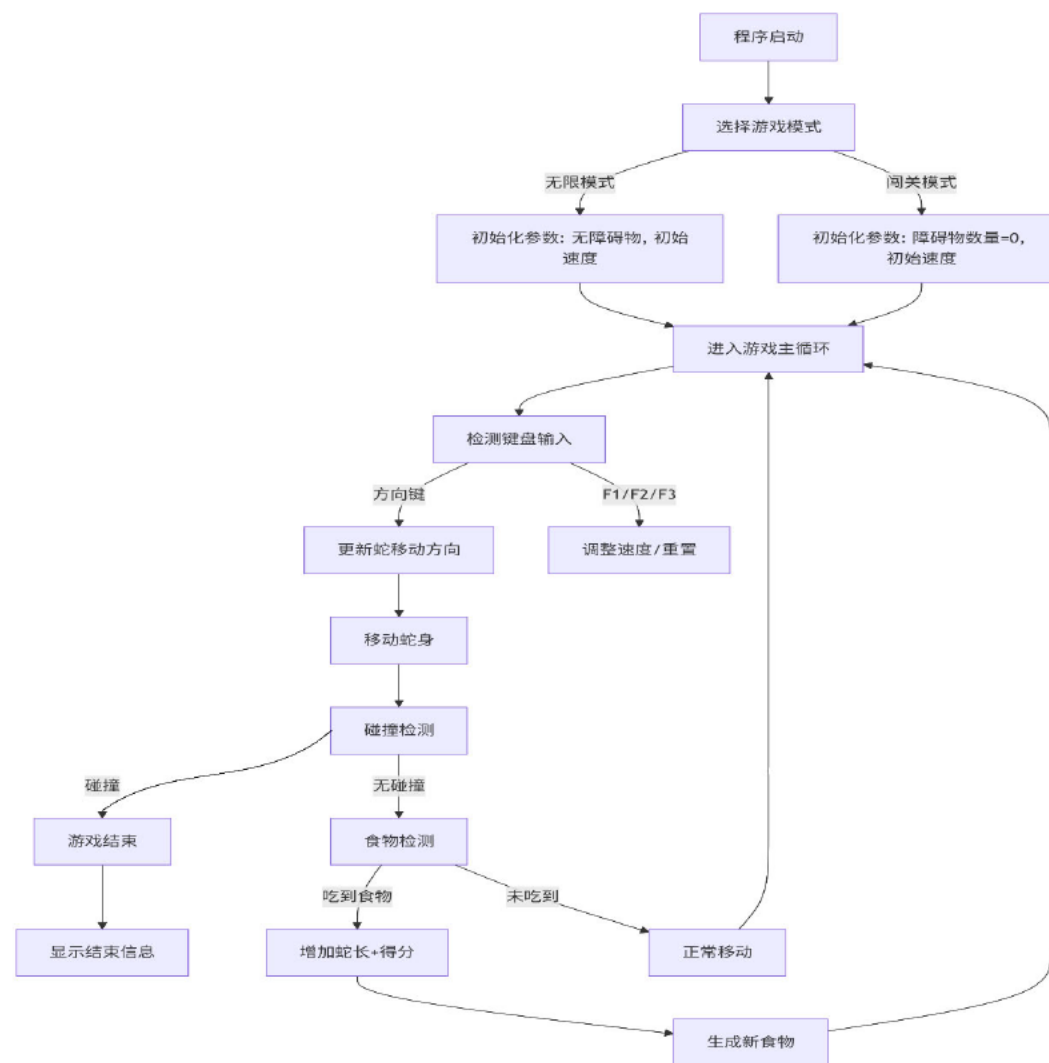
优点

蛇身伸缩的空间复杂度较低

链式队列的进队出队的时间复杂度为 $O(1)$ ，比数组的批量移动元素 ($O(n)$) 更高效。

按需分配内存

系统根据蛇身的长度和障碍物的数量动态分配内存，避免了存储空间的浪费。



1. 游戏流程：通过welcometogame、gamecircle、endgame函数实现。首先进入开始界面，玩家可选择无限模式和闯关模式这两种游戏模式。进入游戏后，在控制台左侧显示游戏界面，右侧显示当前模式、得分和食物分值等提示信息，以及游戏操作说明。通过键盘可以控制蛇身移动以及游戏状态。当退出游戏时控制台会打印退出原因和最终得分等信息。

```
// 欢迎界面
void welcometogame() {
    SetColor(COLOR_YELLOW);
    Pos(40, 10);
    printf("请选择游戏模式: ");
    Pos(40, 12);
    printf("1 - 无限模式");
    Pos(40, 13);
    printf("2 - 闯关模式");

    while (1) {
        if (GetAsyncKeyState('1')) {
            gameMode = 1;
            break;
        }
        if (GetAsyncKeyState('2')) {
            gameMode = 2;
            add = 10;
            break;
        }
        Sleep(100);
    }
    system("cls");
}
```

```
// 结束游戏
void endgame() {
    system("cls");
    SetColor(COLOR_YELLOW);
    Pos(24, 12);
    const char* msg[] = {"撞墙失败!", "咬到自己!", "主动退出游戏", "撞到障碍物!"};
    printf("%s", msg[endgamestatus-1]);
    Pos(24, 13);
    printf("最终得分: %d", score);
    if (gameMode == 2) {
        Pos(24, 14);
        printf("达到关卡: Lv. %d", level);
    }
    Pos(24, 15);
    printf("按任意键退出...");
    getch();
    exit(0);
}
```

```
// 游戏主循环
void gamecircle() {
    Pos(64, 15);
    SetColor(COLOR_YELLOW);
    printf("操作说明: ");
    Pos(64, 16);
    printf("方向键控制移动");
    Pos(64, 17);
    printf("ESC退出 空格暂停");
    if (gameMode == 1) {
        Pos(64, 18);
        printf("F1加速 F2减速 F3重置");
    }

    if (gameMode == 2) {
        createObstacles();
        sleeptime = 200;
    }

    status = R;
    while (1) {
        SetColor(COLOR_YELLOW);
        Pos(64, 9);
        printf("当前模式: %s", gameMode == 1 ? "无限模式" : "闯关模式");
        Pos(64, 10);
        printf("得分: %d", score);
        Pos(64, 11);
        if (gameMode == 2) {
            printf("当前关卡: Lv. %d", level);
            Pos(64, 12);
            printf("下一关需: %d分", level * 100);
        }
        else {
            printf("食物分值: %d", add);
        }

        if (GetAsyncKeyState(VK_UP) && status != D) status = U;
        else if (GetAsyncKeyState(VK_DOWN) && status != U) status = D;
        else if (GetAsyncKeyState(VK_LEFT) && status != R) status = L;
        else if (GetAsyncKeyState(VK_RIGHT) && status != L) status = R;
        else if (GetAsyncKeyState(VK_SPACE)) pause();
        else if (GetAsyncKeyState(VK_ESCAPE)) {
            endgamestatus = 3;
            endgame();
        }

        if (gameMode == 1) {
            if (GetAsyncKeyState(VK_F1)) {
                sleeptime = (sleeptime > 50) ? sleeptime - 30 : 50;
                add = (add < 20) ? add + 2 : 20;
            }
            else if (GetAsyncKeyState(VK_F2)) {
                sleeptime = (sleeptime < 350) ? sleeptime + 30 : 350;
                add = (add > 1) ? add - 2 : 1;
            }
            else if (GetAsyncKeyState(VK_F3)) {
                sleeptime = 200;
                add = 10;
            }
        }

        Sleep(sleeptime);
        snakemove();

        if (gameMode == 2) {
            checkLevelUp();
            if (hitObstacle()) {
                endgamestatus = 4;
                endgame();
            }
        }
    }
}
```

2.部分键盘操作：在无尽模式下，F1实现移动加速，两种状态最短间隔时间不超过50ms，并且移速加快每次吃到食物可以获得一定加分。F2实现移动减速，两种状态最长间隔时间不超过350ms，并且移速加快每次吃到食物可以获得一定减分。F3重置为游戏初始状态。

```
if (gameMode == 1) {  
    if (GetAsyncKeyState(VK_F1)) {  
        sleeptime = (sleeptime > 50) ? sleeptime - 30 : 50;  
        add = (add < 20) ? add + 2 : 20;  
    }  
    else if (GetAsyncKeyState(VK_F2)) {  
        sleeptime = (sleeptime < 350) ? sleeptime + 30 : 350;  
        add = (add > 1) ? add - 2 : 1;  
    }  
    else if (GetAsyncKeyState(VK_F3)) {  
        sleeptime = 200;  
        add = 10;  
    }  
}
```

3.关卡升级：通过checkLevelUp函数实现。在闯关模式下每获得100分进入下一关，每过一关障碍物的个数增加3个并且蛇身移动速度加快，从而增加闯关难度，一共有5关。

```
// 关卡升级  
void checkLevelUp() {  
    int required = level * 100;  
    if (score >= required && level < 5) {  
        level++;  
        createObstacles();  
        if (gameMode == 2) {  
            sleeptime = 200 - (level - 1) * 30;  
            if (sleeptime < 50) sleeptime = 50;  
        }  
    }  
}
```

4.Windows控制台API实现图形化交互：我们通过添加头文件” windows.h”实现控制台窗口的图形界面设计，编写Pos函数实现光标位置控制，SetColor函数实现光标颜色设置，同时通过键盘可以控制贪吃蛇的移动以及游戏设置。

```
// 颜色设置函数  
void SetColor(int color) {  
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);  
}  
  
// 光标定位函数  
void Pos(int x, int y) {  
    COORD pos = { x, y };  
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), pos);  
}
```


5. 游戏初始化：
通过 creatMap 函数打印墙壁，并通过 initsnake 函数创建一个四个节点的链式队列初始化蛇身并打印在控制台上。

```
// 创建地图
void creatMap() {
    SetColor(COLOR_WHITE);
    for (int i = 0; i < 58; i += 2) {
        Pos(i, 0);
        printf("墙");
        Pos(i, 26);
        printf("墙");
    }
    for (int i = 1; i < 26; i++) {
        Pos(0, i);
        printf("墙");
        Pos(56, i);
        printf("墙");
    }
}

// 初始化蛇身
void initsnake() {
    SnakeNode* tail = new SnakeNode(28, 12);
    for (int i = 1; i <= 4; i++) {
        head = new SnakeNode(28 + 2 * i, 12, tail);
        tail = head;
    }

    SetColor(COLOR_RED);
    SnakeNode* current = tail;
    while (current) {
        Pos(current->x, current->y);
        printf("■");
        if (current != head) SetColor(COLOR_WHITE);
        current = current->next;
    }
}
```

6. 食物创建：通过 createfood 函数实现。首先创建一个蛇身节点类的对象作为食物，坐标随机指定，并且保证食物节点与蛇身节点不重合，最后以绿色打印在控制台中。

```
// 创建食物
void createfood() {
    while (1) {
        food = new SnakeNode((rand() % 27) * 2 + 2, rand() % 24 + 1);
        int valid = 1;
        SnakeNode* q = head;
        while (q) {
            if (q->x == food->x && q->y == food->y) {
                valid = 0;
                break;
            }
            q = q->next;
        }
        if (valid) break;
        delete food;
    }

    SetColor(COLOR_GREEN);
    Pos(food->x, food->y);
    printf("■");
}
```


7.障碍物创建：
障碍物只在闯关模式中出现，通过createObstacles函数进行创建。首先进行初始化，删除所有已存在的障碍物节点。然后根据关卡数设置障碍物数：障碍物数量=(关卡数-1)

```
// 创建障碍物
void createObstacles() {
    while (obs_head) {
        Obstacle* tmp = obs_head;
        obs_head = obs_head->next;
        delete tmp;
    }

    int obs_num = (level - 1) * 3;
    if (obs_num > 12) obs_num = 12;

    for (int i = 0; i < obs_num; i++) {
        Obstacle* new_obs = nullptr;
        do {
            int x = (rand() % 27) * 2 + 2;
            int y = rand() % 24 + 1;
            new_obs = new Obstacle(x, y);

            bool valid = true;
            SnakeNode* p = head;
            while (p) {
                if (p->x == x && p->y == y) valid = false;
                p = p->next;
            }
            if (food && food->x == x && food->y == y) valid = false;
            if (x <= 0 || x >= 56 || y <= 0 || y >= 26) valid = false;

            Obstacle* exist = obs_head;
            while (exist) {
                if (exist->x == x && exist->y == y) valid = false;
                exist = exist->next;
            }

            if (!valid) {
                delete new_obs;
                new_obs = nullptr;
            }
        } while (!new_obs);

        new_obs->next = obs_head;
        obs_head = new_obs;
        SetColor(COLOR_BLUE);
        Pos(new_obs->x, new_obs->y);
        printf("■");
    }
}
```

8.蛇身移动：通过snakemove和moveSnake两个函数实现。通过之前键盘控制可以得到贪吃蛇当前的移动方向，并计算出下一时刻头部的坐标。根据这个坐标创建下一时刻头部的蛇身节点，如果吃到食物，直接将新的头部节点插入队列并打印在控制台上即可，并生成下一个食物节点。如果未吃到食物，将新的头部节点插入队列后并将尾部节点出队，在原来尾部打印一个“空白”，视觉上表现出删除效果，避免屏幕闪烁。最后再打印在控制台上，即可得到移动后的蛇身。

```
// 移动处理
void moveSnake(int newX, int newY) {
    SnakeNode* nexthead = new SnakeNode(newX, newY, head);

    if (newX == food->x && newY == food->y) {
        head = nexthead;
        score += add;
        SetColor(COLOR_RED);
        Pos(head->x, head->y);
        printf("■");
        createFood();
    } else {
        head = nexthead;
        SetColor(COLOR_RED);
        Pos(head->x, head->y);
        printf("■");

        SnakeNode* current = head->next;
        while (current) {
            SetColor(COLOR_WHITE);
            Pos(current->x, current->y);
            printf(" ");
            current = current->next;
        }

        SnakeNode* q = head;
        while (q->next->next) q = q->next;
        Pos(q->next->x, q->next->y);
        printf(" ");
        delete q->next;
        q->next = nullptr;
    }
}
```

```
// 蛇移动主逻辑
void snakemove() {
    cantcrosswall();
    switch (status) {
        case U: moveSnake(head->x, head->y - 1); break;
        case D: moveSnake(head->x, head->y + 1); break;
        case L: moveSnake(head->x - 2, head->y); break;
        case R: moveSnake(head->x + 2, head->y); break;
    }
    if (biteself()) {
        endgamestatus = 2;
        endgame();
    }
}
```

9. 碰撞检测：通过 biteself、cantcrosswall、hitObstacle三个函数实现。通过检测贪吃蛇的头部节点是否与其他蛇身节点、障碍物节点、墙壁的坐标重合，判断是否发生碰撞，以此决定是否结束游戏。

```
// 自噬检测
int biteself() {
    SnakeNode* self = head->next;
    while (self) {
        if (self->x == head->x && self->y == head->y) return 1;
        self = self->next;
    }
    return 0;
}
```

```
// 撞墙检测
void cantcrosswall() {
    if (head->x <= 0 || head->x >= 56 || head->y <= 0 || head->y >= 26) {
        endgamestatus = 1;
        endgame();
    }
}

// 障碍物碰撞检测
int hitObstacle() {
    Obstacle* p = obs_head;
    while (p) {
        if (head->x == p->x && head->y == p->y) return 1;
        p = p->next;
    }
    return 0;
}
```



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

3

测试与结论

开始游戏

请选择游戏模式：

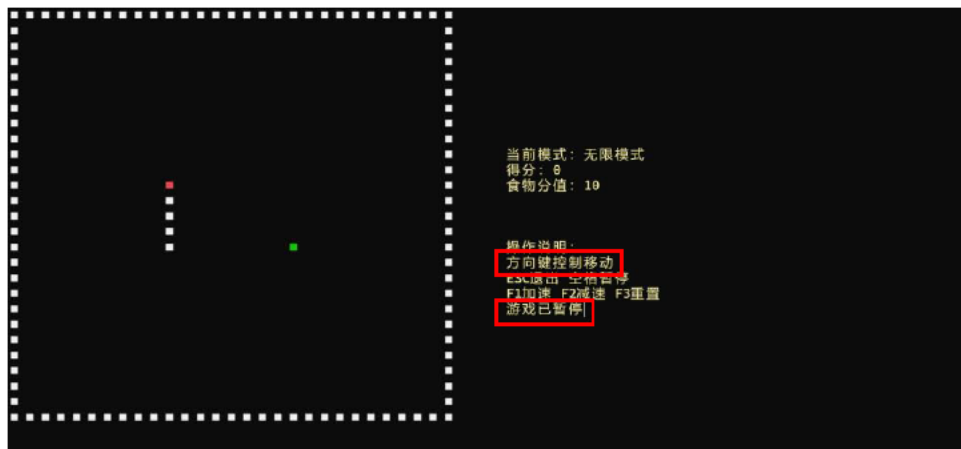
- 1 - 无限模式
- 2 - 闯关模式

测试中发现，贪吃蛇每隔固定的时间都会向前移动一个单位，通过键盘可以改变移动的方向以及游戏的退出、暂停与继续。

选择无限模式的游戏界面



贪吃蛇吃到食物后长度变长，并且游戏得分增加

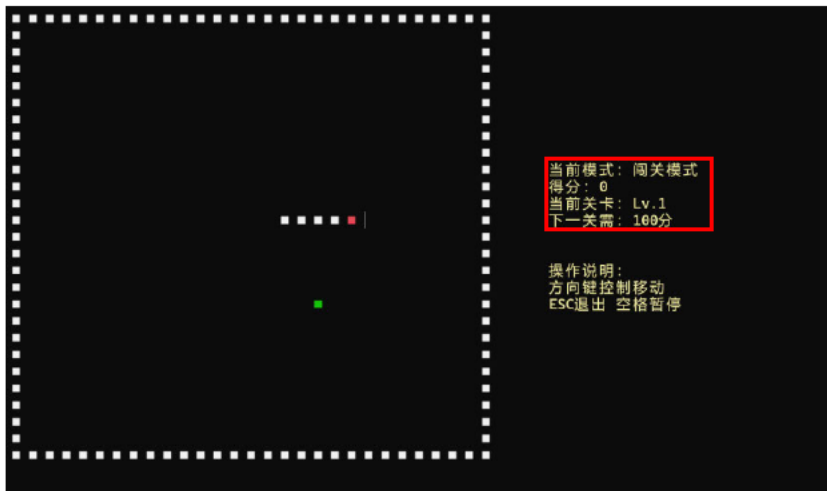
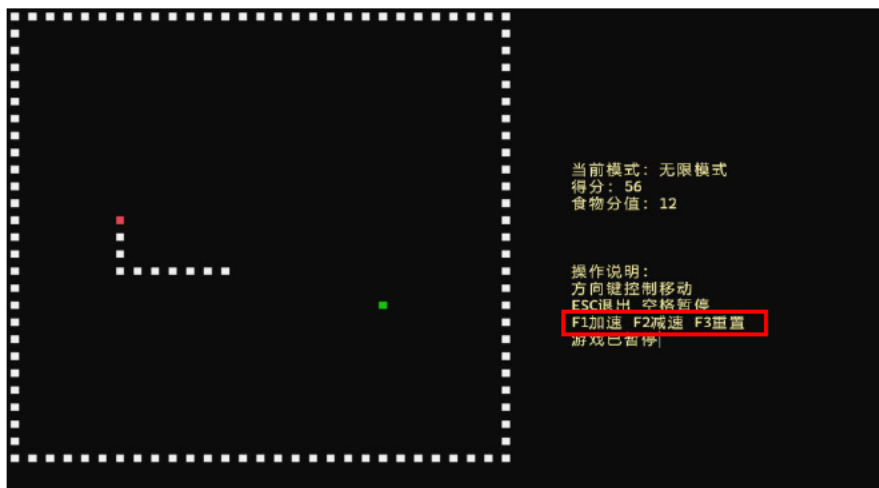


程序运行结果



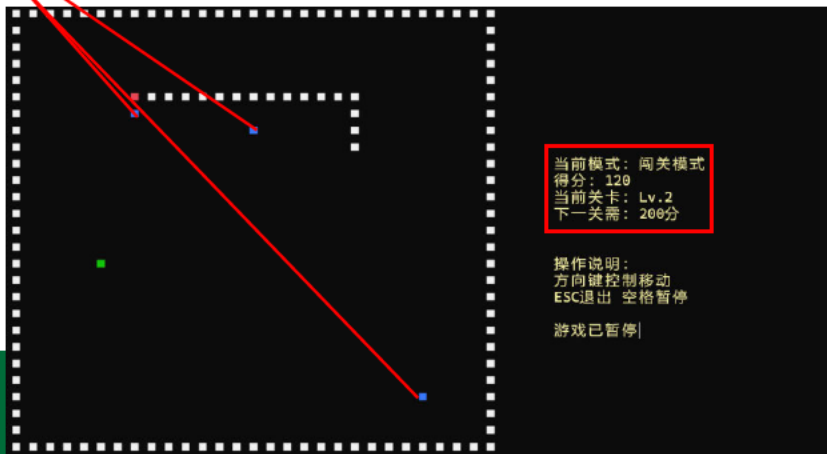
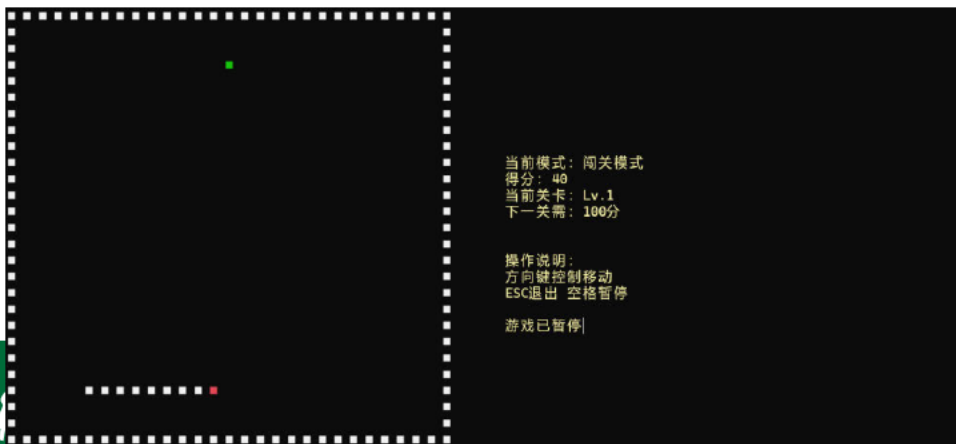
在无限模式下快捷键F1、F2、F3可以控制蛇身移动速度并且改变食物分值，相应提示在控制台右侧

选择闯关模式后的游戏界面，控制台右侧标明了关卡信息



闯关模式大部分操作与无限模式相同，但每过一关障碍物个数增加3个并且蛇身移动速度加快

障碍物



游戏在贪吃蛇撞墙、撞上障碍物或者咬到自己时结束，控制台会打印相关提示信息

```
撞墙失败！  
最终得分：50  
达到关卡：Lv.1  
按任意键退出...  
D:\lessons_and_papers\computer_lessons\C_plus_plus\Codes\codes\x64\Debug\codes.exe (进程 32512)已退出，代码为 0 (0x0)。  
按任意键关闭此窗口。 . . |
```

撞墙

```
撞到障碍物！  
最终得分：120  
达到关卡：Lv.2  
按任意键退出...  
D:\lessons_and_papers\computer_lessons\C_plus_plus\Codes\codes\x64\Debug\codes.exe (进程 30196)已退出，代码为 0 (0x0)。  
按任意键关闭此窗口。 . . |
```

撞到障碍物

```
咬到自己！  
最终得分：80  
按任意键退出...  
D:\lessons_and_papers\computer_lessons\C_plus_plus\Codes\codes\x64\Debug\codes.exe (进程 33712)已退出，代码为 0 (0x0)。  
按任意键关闭此窗口。 . . |
```

撞到自己

- 通过上述测试发现，程序基本满足贪吃蛇游戏的基本要求。在游戏中玩家可以选择闯关模式和无线模式，分别对应两种不同玩法。玩家通过键盘控制贪吃蛇的游动方向和游戏状态，能随机产生食物并且贪吃蛇吃到食物后蛇身长度和游戏分数增加。游戏具有完善的碰撞检测功能，以此判断退出游戏的时机。游戏还给玩家提供了充分的操作提示，进一步提高了玩家的体验感。



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

4 总结与思考

题目难点要点

1.链表的动态管理：在程序设计我们运用了链式队列和单向链表两种数据结构，这两种数据结构都涉及对指针的操作和管理，需要我们特别注意。

2.控制台交互的设计：通过添加“windows.h”头文件实现控制台的交互，从而对游戏界面进行设计。

本组工作特点

1.游戏功能扩展：在游戏中我们设计了无限模式和闯关模式两种模式，给予玩家更多选择。

2.玩家体验感增强：蛇头、蛇身、食物、障碍物标明不同颜色，区分各游戏元素。**蛇身移动过程中，用底色覆盖模拟蛇身运动，避免屏幕刷新与闪烁。**支持快捷键进行暂停、退出以及调整蛇身运动速度的操作，进一步提升玩家的体验感。

3.严谨的内存管理：在贪吃蛇移动式动态增删节点，防止内存泄漏。在切换关卡时释放旧障碍物链表。

本组改进方向

1.添加文件的存取功能，有利于保存之前的游戏进度。

2.引入特殊的食物和障碍物，增强游戏的趣味性。

3.为游戏增加音效系统，提升玩家体验感。



感谢观看

Thanks for Your Attention